

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА
ФРАНКА**

**Факультет прикладної математики та інформатики
Кафедра програмування**

**Лабораторна робота №5
Створення DLL**

Виконала:
Студентка групи ПМІ-46
Тригуб Марта

Мета роботи

Ознайомитися з принципами створення динамічних бібліотек DLL мовою C++, навчитися проектувати, програмувати та тестувати функції бібліотеки, виконати реалізацію функцій зі стандартними типами C++ та типами Windows, а також зібрати DLL для подальшого використання в інших програмах.

Постановка завдання

Відповідно до завдання було розроблено набір із 5 незалежних нетривіальних функцій для обробки числових масивів і текстових даних. Функції підбрано так, щоб у параметрах були використані типи `int`, `double`, `int*`, `bool`, `char`, `char*`.

Було сформульовано такі функції:

1. `CountAboveThreshold` — підрахунок кількості елементів масиву, що більші за заданий поріг.
2. `AverageOfPositive` — обчислення середнього арифметичного додатних елементів масиву.
3. `IsArithmeticProgression` — перевірка, чи утворюють три числа арифметичну прогресію.
4. `ToUpperChar` — переведення символу в верхній регістр.
5. `CountWordsStartingWithLetter` — підрахунок кількості слів у рядку, які починаються із заданої літери.

Реалізація функцій зі стандартними типами C++

```

1  ✓ #include <iostream>
2  | #include <cstring>
3  | using namespace std;
4
5  // Прототипи функцій
6  int CountAboveThreshold(int* arr, int size, int threshold);
7  double AverageOfPositive(double* arr, int size);
8  bool IsArithmeticProgression(double a, double b, double c);
9  char ToUpperChar(char ch);
10 int CountWordsStartingWithLetter(char* text, char letter);
11
12 // 1. Кількість елементів масиву, більших за popir
13 ✓ int CountAboveThreshold(int* arr, int size, int threshold) {
14 |     int count = 0;
15 |     for (int i = 0; i < size; i++) {
16 |         if (arr[i] > threshold) {
17 |             count++;
18 |         }
19 |     }
20 |     return count;
21 | }
22
23 // 2. Середнє арифметичне додатних елементів
24 ✓ double AverageOfPositive(double* arr, int size) {
25 |     double sum = 0.0;
26 |     int count = 0;
27
28 |     for (int i = 0; i < size; i++) {
29 |         if (arr[i] > 0) {
30 |             sum += arr[i];
31 |             count++;
32 |         }
33 |     }
34
35 |     if (count == 0) {
36 |         return 0.0;
37 |     }
38
39 |     return sum / count;
40 | }

```

```

42 // 3. Перевірка арифметичної прогресії
43 bool IsArithmeticProgression(double a, double b, double c) {
44     return (b - a) == (c - b);
45 }
46
47 // 4. Переведення символу в верхній регістр
48 char ToUpperChar(char ch) {
49     if (ch >= 'a' && ch <= 'z') {
50         return ch - 32;
51     }
52     return ch;
53 }
54
55 // 5. Підрахунок слів, що починаються з заданої літери
56 int CountWordsStartingWithLetter(char* text, char letter) {
57     int count = 0;
58     bool newWord = true;
59
60     for (int i = 0; text[i] != '\0'; i++) {
61         if (text[i] == ' ') {
62             newWord = true;
63         }
64         else {
65             if (newWord) {
66                 char first = text[i];
67                 if (first >= 'A' && first <= 'Z') {
68                     first = first + 32;
69                 }
70
71                 char needed = letter;
72                 if (needed >= 'A' && needed <= 'Z') {
73                     needed = needed + 32;
74                 }
75
76                 if (first == needed) {
77                     count++;
78                 }
79                 newWord = false;
80             }
81         }
82     }
83
84     return count;
85 }

```

```

87  int main() {
88      cout << "==== Main1.cpp =====< endl;
89      // Тест 1
90      int arr1[] = { 3, 7, 10, 2, 8, 1 };
91      int size1 = 6;
92      cout << "CountAboveThreshold = "
93          << CountAboveThreshold(arr1, size1, 5) << endl;
94
95      // Тест 2
96      double arr2[] = { -2.5, 3.0, 4.5, -1.0, 2.5 };
97      int size2 = 5;
98      cout << "AverageOfPositive = "
99          << AverageOfPositive(arr2, size2) << endl;
100
101      // Тест 3
102      cout << "IsArithmeticProgression(2, 4, 6) = "
103          << IsArithmeticProgression(2, 4, 6) << endl;
104      cout << "IsArithmeticProgression(1, 3, 8) = "
105          << IsArithmeticProgression(1, 3, 8) << endl;
106
107      // Тест 4
108      cout << "ToUpperChar('b') = "
109          << ToUpperChar('b') << endl;
110
111      // Тест 5
112      char text[] = "Apple banana apricot Berry avocado";
113      cout << "CountWordsStartingWithLetter(text, 'a') = "
114          << CountWordsStartingWithLetter(text, 'a') << endl;
115
116      return 0;
117  }

```

Реалізація та тестування функцій зі стандартними типами C++

```

==== Main1.cpp =====
CountAboveThreshold = 3
AverageOfPositive = 3.33333
IsArithmeticProgression(2, 4, 6) = 1
IsArithmeticProgression(1, 3, 8) = 0
ToUpperChar('b') = B
CountWordsStartingWithLetter(text, 'a') = 3

```

Результати тестування функцій зі стандартними типами C++

Реалізація функцій з використанням типів Windows

```

1  #include <windows.h>
2  #include <iostream>
3  using namespace std;
4
5  // Прототипы
6  INT CountAboveThresholdWin(INT* arr, INT size, INT threshold);
7  DOUBLE AverageOfPositiveWin(DOUBLE* arr, INT size);
8  BOOL IsArithmeticProgressionWin(DOUBLE a, DOUBLE b, DOUBLE c);
9  CHAR ToUpperCharWin(CHAR ch);
10 INT CountWordsStartingWithLetterWin(LPSTR text, CHAR letter);
11
12 // 1
13 INT CountAboveThresholdWin(INT* arr, INT size, INT threshold) {
14     INT count = 0;
15     for (INT i = 0; i < size; i++) {
16         if (arr[i] > threshold) {
17             count++;
18         }
19     }
20     return count;
21 }
22
23 // 2
24 DOUBLE AverageOfPositiveWin(DOUBLE* arr, INT size) {
25     DOUBLE sum = 0.0;
26     INT count = 0;
27
28     for (INT i = 0; i < size; i++) {
29         if (arr[i] > 0) {
30             sum += arr[i];
31             count++;
32         }
33     }
34
35     if (count == 0) {
36         return 0.0;
37     }
38
39     return sum / count;
40 }
41
42 // 3
43 BOOL IsArithmeticProgressionWin(DOUBLE a, DOUBLE b, DOUBLE c) {
44     return ((b - a) == (c - b)) ? TRUE : FALSE;
45 }
46

```

```

47 // 4
48 v CHAR ToUpperCharWin(CHAR ch) {
49 v     if (ch >= 'a' && ch <= 'z') {
50         return ch - 32;
51     }
52     return ch;
53 }
54
55 // 5
56 v INT CountWordsStartingWithLetterWin(LPSTR text, CHAR letter) {
57     INT count = 0;
58     BOOL newWord = TRUE;
59
60     for (INT i = 0; text[i] != '\0'; i++) {
61         if (text[i] == ' ') {
62             newWord = TRUE;
63         }
64         else {
65             if (newWord) {
66                 CHAR first = text[i];
67                 if (first >= 'A' && first <= 'Z') {
68                     first = first + 32;
69                 }
70
71                 CHAR needed = letter;
72                 if (needed >= 'A' && needed <= 'Z') {
73                     needed = needed + 32;
74                 }
75
76                 if (first == needed) {
77                     count++;
78                 }
79                 newWord = FALSE;
80             }
81         }
82     }
83
84     return count;
85 }
86

```

```

87  int main() {
88      cout << "==== Main2.cpp =====> endl;
89      INT arr1[] = { 3, 7, 10, 2, 8, 1 };
90      INT size1 = 6;
91      cout << "CountAboveThresholdWin = "
92           << CountAboveThresholdWin(arr1, size1, 5) << endl;
93
94      DOUBLE arr2[] = { -2.5, 3.0, 4.5, -1.0, 2.5 };
95      INT size2 = 5;
96      cout << "AverageOfPositiveWin = "
97           << AverageOfPositiveWin(arr2, size2) << endl;
98
99      cout << "IsArithmeticProgressionWin(2, 4, 6) = "
100         << IsArithmeticProgressionWin(2, 4, 6) << endl;
101
102      cout << "IsArithmeticProgressionWin(1, 3, 8) = "
103         << IsArithmeticProgressionWin(1, 3, 8) << endl;
104
105      cout << "ToUpperCharWin('b') = "
106         << ToUpperCharWin('b') << endl;
107
108      CHAR text[] = "Apple banana apricot Berry avocado";
109      cout << "CountWordsStartingWithLetterWin(text, 'a') = "
110         << CountWordsStartingWithLetterWin(text, 'a') << endl;
111
112      return 0;
113  }

```

Реалізація та тестування функцій із типами Windows

```

==== Main2.cpp =====
CountAboveThresholdWin = 3
AverageOfPositiveWin = 3.33333
IsArithmeticProgressionWin(2, 4, 6) = 1
IsArithmeticProgressionWin(1, 3, 8) = 0
ToUpperCharWin('b') = B
CountWordsStartingWithLetterWin(text, 'a') = 3

```

Результати тестування функцій із типами Windows

У ході виконання роботи було виконано заміну стандартних типів C++ на відповідні типи Windows:

- int -> INT
- double -> DOUBLE

- bool -> BOOL
- char -> CHAR
- char* -> LPSTR

Результати тестування повністю співпали з результатами, отриманими для реалізації зі стандартними типами.

Створення DLL

Для створення динамічної бібліотеки було створено окремий проєкт типу Dynamic Link Library (DLL). Для експорту функцій використано конструкцію `extern "C" __declspec(dllexport)`, що забезпечує коректне експортування функцій із бібліотеки.

```

1  #include "pch.h"
2  #include <windows.h>
3
4  extern "C" __declspec(dllexport) INT CountAboveThresholdWin(INT* arr, INT size, INT threshold) {
5      INT count = 0;
6      for (INT i = 0; i < size; i++) {
7          if (arr[i] > threshold) {
8              count++;
9          }
10     }
11     return count;
12 }
13
14 extern "C" __declspec(dllexport) double AverageOfPositiveWin(double* arr, INT size) {
15     double sum = 0.0;
16     INT count = 0;
17
18     for (INT i = 0; i < size; i++) {
19         if (arr[i] > 0) {
20             sum += arr[i];
21             count++;
22         }
23     }
24
25     if (count == 0) {
26         return 0.0;
27     }
28
29     return sum / count;
30 }
31
32 extern "C" __declspec(dllexport) BOOL IsArithmeticProgressionWin(double a, double b, double c) {
33     return ((b - a) == (c - b)) ? TRUE : FALSE;
34 }
35
36 extern "C" __declspec(dllexport) CHAR ToUpperCharWin(CHAR ch) {
37     if (ch >= 'a' && ch <= 'z') {
38         return ch - 32;
39     }
40     return ch;
41 }
42

```

```

42
43 extern "C" __declspec(dllexport) INT CountWordsStartingWithLetterWin(LPSTR text, CHAR letter) {
44     INT count = 0;
45     BOOL newWord = TRUE;
46
47     for (INT i = 0; text[i] != '\0'; i++) {
48         if (text[i] == ' ') {
49             newWord = TRUE;
50         }
51         else if (newWord) {
52             CHAR first = text[i];
53             if (first >= 'A' && first <= 'Z') {
54                 first += 32;
55             }
56
57             CHAR needed = letter;
58             if (needed >= 'A' && needed <= 'Z') {
59                 needed += 32;
60             }
61
62             if (first == needed) {
63                 count++;
64             }
65
66             newWord = FALSE;
67         }
68     }
69
70     return count;
71 }

```

Код експортованих функцій динамічної бібліотеки DLL

Після успішної компіляції в папці Debug було отримано файли:

- *.dll
- *.lib

 Lab5_DLL1.dll	22.03.2026 20:18	Розширення заст...	60 КБ
 Lab5_DLL1.exp	22.03.2026 20:18	Exports Library File	2 КБ
 Lab5_DLL1.lib	22.03.2026 20:18	Object File Library	3 КБ

Отримані файли DLL та LIB після збірки проєкту

Висновки

У ході виконання лабораторної роботи було спроектовано та реалізовано 5 незалежних функцій для обробки числових і текстових даних. Спочатку функції були реалізовані з використанням стандартних типів C++, після чого виконано їх адаптацію до типів Windows. Проведене тестування показало, що результати роботи обох реалізацій повністю збігаються. На завершальному етапі було створено динамічну бібліотеку DLL, у якій реалізовані експортовані функції. У результаті отримано файли dll та lib, придатні для подальшого використання в інших програмах.